

AI Usage DISCLOSURE FORM

1. Team Details

Team Name: GS CodeStorm

Project / Product Name: Streaming Live RAG

Organization / Institution (if any): VIT university, vellore

Submission Date: 30/9/2026

2. AI Usage Declaration

Did your team use any Artificial Intelligence (AI) in developing this project? Yes / No

YES

3. Purpose of AI Usage (Brief Details)

Idea generation / brainstorming:

AI was used to brainstorm the concept of state-preserving RAG, streaming responses, late-constraint refinement, multi-intent handling, and possible use cases.

Code generation or assistance:

AI was used to assist with Python, Flask, RAG pipeline, retrieval, session-state management, SSE streaming, API integration, testing, and debugging code.

UI / UX design:

AI was used to suggest improvements to the web interface, chat flow, dashboard layout, and user interaction design.

Content creation:

AI was used to assist in preparing project documentation, README content, presentation material, demo script, architecture explanations, and project descriptions.

Data analysis:

AI was used to assist with interpreting retrieval outputs, identifying irrelevant retrieved evidence, and designing evaluation metrics for the RAG system.

Testing / debugging:

AI was used to identify potential implementation issues, suggest test cases, troubleshoot errors, and improve retrieval, grounding, streaming, and state-preservation logic.

Other:

AI was used for technical explanations, implementation guidance, code review, and refinement of project documentation.

4. Feature Origin Classification (Please add/delete based on the number of the feature)**Feature 1**

Feature Name: State-Preserving Conversation Management

Self-Generated/AI-Generated/Both: Both

Description:

The feature was designed by the team based on the Theme 4 requirement of state-preserving answer refinement. AI tools were used to brainstorm implementation approaches and provide coding assistance. The team implemented and modified the conversation state mechanism to retain the original query, retrieved evidence, constraints, and previous answer versions. The final implementation was reviewed and integrated by the team.

Feature 2

Feature Name: Late-Constraint Answer Refinement

Self-Generated/AI-Generated/Both: Both

Description:

The team designed the concept of refining an existing answer when the user adds a new constraint instead of restarting the complete conversation. AI assistance was used for implementation suggestions and debugging. The final system combines the original intent with the new constraint, performs targeted retrieval, and generates a refined answer while preserving the previous state.

Feature 3

Feature Name: Hybrid Retrieval – FAISS + BM25 + RRF

Self-Generated/AI-Generated/Both: Both

Description:

The team selected hybrid retrieval to improve both semantic and keyword-based retrieval. AI tools assisted with implementation guidance and debugging for FAISS vector search, BM25 search, and Reciprocal Rank Fusion (RRF). The retrieved results were integrated, tested, and modified by the team according to the project's requirements.

Feature 4

Feature Name: Multi-Intent Decomposition

Self-Generated/AI-Generated/Both: Both

Description:

AI assistance was used to explore approaches for identifying multiple intents within a single user query. The team implemented an intent decomposition component that separates complex queries into manageable intents before retrieval and answer generation. The implementation was modified and tested according to the project workflow.

Feature 5

Feature Name: Evidence Filtering and Grounded Generation

Self-Generated/AI-Generated/Both: Both

Description:

The team designed the evidence-aware generation approach to reduce irrelevant information in responses. AI assistance was used for implementation suggestions and debugging. Retrieved evidence is filtered before generation, and the generated answer is checked against available evidence. The final logic was implemented, tested, and modified by the team.

Feature 6

Feature Name: Live Streaming using Server-Sent Events (SSE)

Self-Generated/AI-Generated/Both: Both

Description:

The team implemented live response streaming using Flask and Server-Sent Events. AI tools were used to assist with understanding SSE implementation, debugging streaming behavior, and improving the frontend integration. The final streaming workflow was integrated and tested by the team.

Feature 7

Feature Name: Answer Versioning

Self-Generated/AI-Generated/Both: Both

Description:

The answer versioning concept was developed to represent the evolution of an answer as the conversation continues. AI assistance was used during implementation and debugging. The system maintains answer versions such as v1, v2, and v3 so that refinement can occur without losing the previous conversational state.

Feature 8

Feature Name: Dynamic RAG Controller

Self-Generated/AI-Generated/Both: Both

Description:

The team designed a controller with states such as WAIT, PROCESS, RETRIEVE, and REFINE. AI tools assisted with implementation ideas and debugging. The final controller determines the appropriate processing path based on the user's current request and conversation state.

5. Ethical & Compliance Confirmation

AI usage complies with guidelines and policies. **Yes**

No proprietary or copyrighted data misused. **I Agree**

6. Declaration & Sign-Off

Name of Team Representative: Santhosh S

Role: Team Representative / Project Developer

Signature: santhosh.s

Date: 30| 09| 2026